
Cloud-Native Trail

A Structured Roadmap for Building Cloud-Native Systems

1. Containerization

Purpose

Containerization packages applications with their dependencies to ensure consistency across development, testing, and production environments.

Key Concepts

- Applications run in isolated environments
- Portable across platforms and infrastructure
- Immutable infrastructure pattern

Tools & Technologies

- **Docker**
 - Most common container build and packaging tool
 - Supports multi-stage builds for smaller images
- **Container Images**
 - Can include any runtime or legacy system (e.g., PDP-11 via emulation)
 - Versioned and immutable
- **Microservices Architecture**
 - Monolithic apps should gradually be decomposed
 - New functionality should be written as independent services

Best Practices

- One process per container
 - Minimal base images (Alpine, Distroless)
 - Externalize configuration
-

2. Continuous Integration & Continuous Delivery (CI/CD)

Purpose

Automate the build, test, and deployment of applications to reduce risk and increase delivery speed.

Core Capabilities

- Automated builds on every code change
- Automated testing (unit, integration, security)
- Automated deployments to staging and production

CI/CD Workflow

1. Code pushed to Git repository
2. Pipeline builds container image
3. Tests executed automatically
4. Image pushed to registry
5. Deployment triggered via GitOps

Tools & Technologies

- **CI Systems:** GitHub Actions, GitLab CI, Jenkins
- **CD & GitOps:**
 - **Argo CD** – declarative Kubernetes deployments
 - **Argo Workflows** – batch jobs and pipelines
 - **Argo Rollouts** – blue/green and canary deployments

Best Practices

- Automated rollbacks on failure
 - Progressive delivery
 - Infrastructure as Code (IaC)
-

3. Orchestration & Application Definition

Purpose

Manage containerized applications at scale across clusters of machines.

Core Platform

- **Kubernetes**
 - Scheduling
 - Scaling
 - Self-healing
 - Service abstraction

Kubernetes Distributions

Choose a **CNCF-certified Kubernetes** platform:

- Managed: EKS, GKE, AKS
- Self-managed: kubeadm, Rancher, OpenShift
- Reference list: <https://cncf.io/ck>

Application Packaging

- **Helm**
 - Templated Kubernetes manifests
 - Versioned application releases
 - Simplifies upgrades and rollbacks

Best Practices

- Declarative configuration
 - Namespace isolation
 - Resource requests and limits
-

4. Observability & Analysis

Purpose

Gain visibility into system behavior and performance.

Three Pillars of Observability

1. **Metrics**
2. **Logs**
3. **Traces**

Tools & Technologies

- **Prometheus**
 - Metrics collection and alerting
- **Fluentd**
 - Centralized logging
- **Jaeger**
 - Distributed tracing
 - OpenTracing compatible

Best Practices

- Use structured logging
 - Define Service Level Indicators (SLIs)
 - Alert on symptoms, not causes
-

5. Service Proxy, Discovery & Mesh

Purpose

Manage service-to-service communication in microservice architectures.

Service Discovery

- **CoreDNS**
 - DNS-based service discovery inside Kubernetes
 - Flexible and extensible

Service Mesh

- **Envoy**
 - High-performance L7 proxy
- **Linkerd**
 - Lightweight, Kubernetes-native service mesh

Capabilities

- Traffic routing
- Load balancing
- Retries and timeouts
- Mutual TLS (mTLS)

- Health checks

Best Practices

- Use mesh only when complexity justifies it
 - Start with observability features
-

6. Networking, Policy & Security

Purpose

Secure and control how workloads communicate and behave.

Networking

- **CNI Plugins**
 - Calico (policy-driven)
 - Flannel (simple networking)
 - Weave Net (mesh-style networking)

Policy Enforcement

- **Open Policy Agent (OPA)**
 - Admission control
 - Authorization
 - Compliance rules

Runtime Security

- **Falco**
 - Detects anomalous behavior
 - Monitors system calls

Best Practices

- Zero Trust networking
 - Network policies by default
 - Least privilege access
-

7. Distributed Databases & Storage

Purpose

Enable scalable, resilient data storage for cloud-native workloads.

Distributed Databases

- **Vitess**
 - MySQL sharding and scaling
- **TiKV**
 - Distributed transactional key-value store
 - Written in Rust

Storage Orchestration

- **Rook**
 - Manages Ceph and other storage backends in Kubernetes

Cluster State Storage

- **etcd**
 - Strongly consistent key-value store
 - Kubernetes control plane backbone

Best Practices

- Separate data plane and control plane
 - Backup and disaster recovery planning
-

8. Streaming & Messaging

Purpose

Enable asynchronous, high-performance communication.

Communication Models

- Request/Reply
- Publish/Subscribe

- Event-driven architectures

Tools & Technologies

- **gRPC**
 - High-performance RPC
 - Strongly typed APIs
- **NATS**
 - Lightweight messaging
 - Pub/Sub and queueing
- **CloudEvents**
 - Standard event format across platforms

Best Practices

- Prefer async communication where possible
 - Design for eventual consistency
-

9. Container Registry & Runtime

Purpose

Securely store and run container images.

Container Registries

- **Harbor**
 - Image signing
 - Vulnerability scanning
 - Role-based access control

Container Runtimes

- **containerd**
- **CRI-O**
 - Both OCI-compliant
 - Kubernetes-native runtimes

Best Practices

- Scan images for vulnerabilities

- Use signed images only
 - Minimize base images
-

10. Software Distribution & Supply Chain Security

Purpose

Ensure software authenticity and integrity.

Tools & Standards

- **Notary**
 - Implements The Update Framework (TUF)
 - Cryptographic signing of artifacts

Best Practices

- Secure the software supply chain
 - Enforce signature verification
 - Track provenance and SBOMs
-

Summary View

Layer	Focus
Build	Containers, CI
Deploy	Kubernetes, Helm, Argo
Observe	Metrics, Logs, Traces
Connect	Service Mesh, Networking
Secure	Policy, Runtime Security
Store	Databases, Storage
Communicate	Messaging, Events
Distribute	Registry, Signing
